

AI Maze 寻路规则详解

本文档描述 CompositeAgent (组合决策器) 控制下, **局部贪心与全局规划**的完整运作规则, 以及两者之
坐标系: (row, col), 左上角为 (1,1), 向下 row 增大, 向右 col
增大。

一、游戏基础规则

格子类型	符号	效果
起点	S	玩家出生点, BOSS 失败后回到此处
终点	E	所有 BOSS 击败后到达即通关
金币	C / G	拾取 +50 金币, 格子变为空地
陷阱	T	首次踩踏 -30 金币, 格子变为空地 (只触发一次)
BOSS	B	到达后进入战斗阶段
墙壁	#	不可进入
空地		可通行, 无效果
锁/机关	L	可通行

评分公式 (越高越好) :

$$\text{score} = \frac{\text{total_value}}{\text{total_steps}}$$

$\text{total_value} = \text{金币数} \times 50 - \text{触发陷阱数} \times 30 - \text{BOSS失败次数} \times \text{CoinConsumption}$

二、视野与迷雾规则 (FOV)

- **视野半径 = 1**, 即玩家每步能看到以当前位置为中心的 **3×3 区域** (共最多 9 格)。
- 每次移动后, 模拟器调用 `_reveal_fov(pos)`, 将玩家新位置周围 3×3 格子的真实内容写入 `fog_map`。
- `fog_map[r][c] = None` 表示该格**从未被视野覆盖过** (黑雾)。
- 已揭露的格子内容**永久记住**, 不会再次变成迷雾。
- AI 能访问的唯一地图是 `fog_map`, **完全无法直接访问** `ground_truth`。

重要推论: - 玩家在 (r, c) 时, 能知道 (r±1, c) 和 (r, c±1) 等直接邻居的内容。
- 但 (r+2, c) 等位置**仍是黑雾**, 必须先走到 (r+1, c) 才能揭露。 -
这意味着: AI **不可能提前知道两步之外是否是死路**, 必须走到近处才能判断。

三、局部贪心策略 (LocalGreedyAgent)

3.1 总体流程

```
1.          3×3
2.          score > 0
3.
4.    score  0  STAY  CompositeAgent
```

3.2 3×3 窗口格子分类

以玩家在 (r, c) 为中心, 窗口内 8 个格子按距离分为两类:

类型	格子	曼哈顿距离	到达方式
直接邻居	上(r-1,c) 下(r+1,c) 左(r,c-1) 右(r,c+1)	dist = 1	直接走过去
对角邻居	(r-1,c-1) (r-1,c+1) (r+1,c-1) (r+1,c+1)	dist = 2	需经过一个中转格

3.3 格子是否纳入评分的条件

直接邻居 (dist=1) : - fog_map[nr][nc] is None (黑雾格) → **纳入评分**, 给予探索奖励

- is_walkable(nr, nc) == True (已知可走格) → **纳入评分**, 按实际内容计算

- 已知为墙 (is_walkable == False 且非 None) → **跳过**

对角邻居 (dist=2) : - 必须 is_walkable(nr, nc) == True (黑雾格不纳入)

- 还必须存在至少一条可通行的中转路径: - 中转格1 (r, nc) (先横后竖) 可走, **或者**

- 中转格2 (nr, c) (先竖后横) 可走 - 以上条件都满足才纳入评分, 否则跳过

为什么对角黑雾格不纳入? 对角格需要 2 步才能到达, 且中转格的内容也可能未知, 路径不确定, 不适合

3.4 评分公式

直接邻居中的黑雾格 (单独处理)

```
score = explore_bonus
       - w_backtrack
```

- explore_bonus = 8.0: 走向未知格子的探索奖励 (固定值, 不除以距离)
- w_backtrack = 2.0: 对上一步位置施加的回头路惩罚

已知可走格（直接邻居 dist=1 或对角邻居 dist=2）

```
raw_value =      : +50
                : -30
                : 0
              /   /   / BOSS : 0

movement_cost = dist * step_cost      # step_cost = 10.0

score = (raw_value - movement_cost) / dist
        - visited_penalty
        - w_backtrack
```

具体数值举例：

格子类型	dist	计算过程	最终得分（无修正项）
金币	1	(50 - 10) / 1	+40.0
金币	2	(50 - 20) / 2	+15.0
陷阱（未触发）	1	(-30 - 10) / 1	-40.0
空地	1	(0 - 10) / 1	-10.0
空地	2	(0 - 20) / 2	-10.0
黑雾（直接邻居）	1	explore_bonus	+8.0

□ **已知空地的得分永远是负数 (-10.0)！** 这是当前评分公式的核心问题：
 当玩家周围只有空地和黑雾时，空地得分 = -10，黑雾得分 = +8，
 AI 会**优先向黑雾方向移动**，而不愿穿越已知空地去探索更深的区域。
 这会导致 AI 在走到已知通道末端时，宁可回头去探索旁边的黑雾，也不继续往前走。

修正项参数

参数	默认值	含义
visited_penalty	3.0	历史走过的格子额外扣分（防止原地打转）
w_backtrack	2.0	上一步所在位置额外扣分（防止来回折返）

3.5 选目标与走第一步

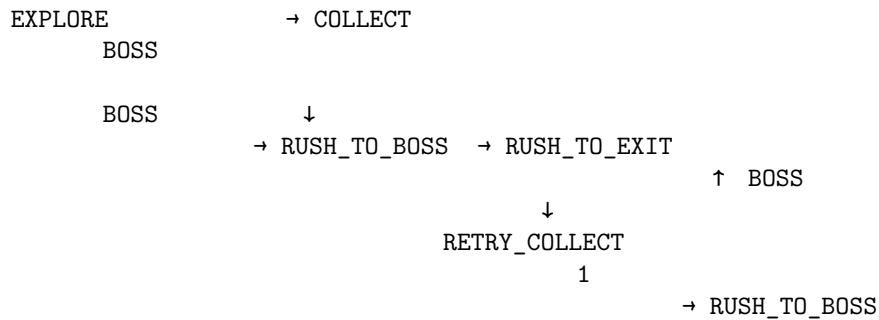
```
candidates.sort(key=score, reverse=True)
for best_pos, best_score in candidates:
    if best_score <= 0:
        break #
    first_step = _first_step(cur_pos, best_pos)
    if first_step is not None:
        return Action(first_step) #

return Action("STAY") # score > 0
```

对角格的中转逻辑 (`_first_step`) : - 对角目标 (`tr, tc`) 需要中转格, 候选中转格为 (`r, tc`) 和 (`tr, c`) - 选**第一个** `is_walkable == True` 的中转格作为本回合第一步
 - 若两个中转格都不可走, 该对角目标不可达, 跳过尝试下一个候选

四、全局规划策略 (GlobalPlannerAgent)

4.1 五阶段状态机



4.2 各阶段行为

EXPLORE (探索阶段)

- **目标**: 揭露尽可能多的地图, 同时寻找 BOSS 位置
- **寻路方法**: `_bfs_to_frontier`——BFS 在**已知格子**中扩展, 找到最近的**前沿格子**
- **前沿格子定义**: 已知可行且至少有一个直接邻居是黑雾 (`fog_map == None`)
- **BFS 约束**: 只经过 `is_walkable == True` 的格子 (严格遵守迷雾, 不走入未知区域)
- **路径内容**: 从当前位置到前沿格子的完整格子列表 (含前沿格子本身)
- **无前沿时**: 已知区域内找不到任何前沿格, 转为 COLLECT 阶段

COLLECT (专项收集阶段)

- **目标**: 收集足够金币 (`retry_buffer × CoinConsumption`)
- **寻路方法**: `_best_coin_target` 选最近金币, 用 Dijkstra 规划路径 (考虑陷阱代价)
- **无金币时**: `fog_map` 中找不到任何金币格, 强制切换到 RUSH_TO_BOSS

RUSH_TO_BOSS (冲向BOSS阶段)

- **目标**: 快速到达已知的 BOSS 位置
- **寻路方法**: Dijkstra + 陷阱代价权重, 自动权衡绕路代价 vs 踩陷阱代价
- **陷阱权重**: 踩一次陷阱的代价 = 1 + `trap_step_cost` 31, 若绕路步数 < 31 则自动绕开
- **BOSS位置未知时**: 回退到 EXPLORE 阶段

RUSH_TO_EXIT (冲向出口阶段)

- **触发条件:** fog_map 中所有已知 BOSS 格均消失 (被击败) 且曾击败过至少一个
- **目标:** 以最短步数到达终点 E
- **寻路方法:** A* (不考虑陷阱, 追求步数最短)
- **不可逆:** 进入此阶段后不会切换回其他阶段

RETRY_COLLECT (失败重试收集阶段)

- **触发条件:** BOSS 战失败 (金币骤降约 CoinConsumption) 且剩余金币 $< \text{CoinConsumption}$
- **目标:** 收集至少 1 次重试所需的金币
- **寻路方法:** 与 COLLECT 相同 (Dijkstra + 陷阱代价)
- **完成条件:** coins $\geq \text{CoinConsumption}$, 切回 RUSH_TO_BOSS

4.3 阶段切换条件详表

当前阶段	切换到	触发条件
EXPLORE	COLLECT	已知区域内无前沿格 (地图全部探索完)
EXPLORE	RUSH_TO_BOSS	发现BOSS + 金币 $\geq$ 缓冲目标 + 探索比例 $\geq$ 40%
COLLECT	RUSH_TO_BOSS	金币 $\geq \text{retry_buffer} \times$ CoinConsumption 且满足探索条件
COLLECT	RUSH_TO_BOSS	地图内无金币可收集 (强制冲)
RUSH_TO_BOSS	RETRY_COLLECT	战斗失败 + 剩余金币 $<$ CoinConsumption
RUSH_TO_BOSS	EXPLORE	BOSS 位置未知 (路径规划失败时回退)
RETRY_COLLECT	RUSH_TO_BOSS	金币 $\geq$ CoinConsumption
任意阶段	RUSH_TO_BOSS	剩余步数 $\leq$ rush_round_threshold 50
任意阶段	RUSH_TO_EXIT	且已知BOSS位置 所有已知BOSS消失 + 曾击败过 ≥ 1 个

4.4 BOSS 位置追踪

- 每回合调用 _scan_boss, 扫描 fog_map 中所有 CELL_BOSS 格子
- 支持多 BOSS 地图: 维护 _all_boss_pos 列表
- BOSS 被击败后格子变为空地, 从列表中自动移除
- 当前追踪目标 _known_boss_pos: 若目标消失则自动切换列表中的下一个

4.5 路径执行机制

全局规划器维护一个路径缓存 `self._path`，每回合从中 `pop(0)` 取出下一步：

```
def decide(ctx):
    #
    _scan_boss / _detect_failure / _detect_boss_defeated / _update_phase

    #
    if not self._path or not ctx.maze.is_walkable(*self._path[0]):
        self._replan(ctx)

    if not self._path:
        return STAY

    next_pos = self._path.pop(0)
    return Action(next_pos)
```

路径全部走完或中途失效时，自动触发 `_replan` 重新规划。

五、局部与全局的切换规则 (CompositeAgent)

5.1 决策优先级 (从高到低)

```
1  CombatAgent
   ↓
2  RUSH  RUSH_TO_BOSS / RUSH_TO_EXIT
   ↓ RUSH
3  _global_active == True
   ↓
4
   ↓ STAY
```

5.2 局部 → 全局 的切换条件

触发条件：局部贪心连续返回 STAY 达到 `stuck_threshold` 2 次

```
1 STAY_stuck_count = 1          STAY
2 STAY_stuck_count = 2  stuck_threshold →
   _global_active = True _stuck_count = 0
   _path = []
```

为什么是 2 次？ `stuck_threshold=2` 意味着局部贪心看不到任何 `score > 0` 的格子，连续 2 次都无法行动，才判定为真正卡住。

注意：局部贪心返回 STAY 并不代表玩家不动，而是 CompositeAgent 第 1 次 STAY 时仍输出 STAY (玩家原地)，第 2

次才切换为全局规划的实际移动指令。

5.3 全局 → 局部 的切换条件

触发条件：全局规划器主导时 (`_global_active == True`)，每回合检查局部贪心的 `_score_3x3` 结果，**只要有任意一个格子** `score > 0` 就立即切回局部。

```
candidates = local_greedy._score_3x3(r, c, ctx.maze)
has_local_reward = any(score > 0 for _, score in candidates)
if has_local_reward:
    _global_active = False
    _stuck_count = 0
    global_planner._path = []    #
    #
```

什么情况下 `score > 0`？ - 视野内有**金币格** (`dist=1` 时 `score=40`, `dist=2` 时 `score=15`) - 视野内有**黑雾直接邻居** (`score = explore_bonus = 8.0`) - 注意：已知空地 `score = -10`，**不会触发切回局部**

5.4 RUSH 阶段不被局部打断

```
if global_planner.phase in (Phase.RUSH_TO_BOSS, Phase.RUSH_TO_EXIT):
    _global_active = True
    _stuck_count = 0
    return global_planner.decide(ctx)    #
```

RUSH 阶段目标明确（冲向特定位置），不应被局部的“顺路拾取”打断，否则会导致路径反复中断、绕路。

5.5 完整决策流程图

```
CompositeAgent.decide(ctx)

[ ] CombatAgent.should_fight(ctx) == True?
    Yes → _stuck_count

[RUSH ] global_planner.phase {RUSH_TO_BOSS, RUSH_TO_EXIT}?
    Yes → _global_active=True,

[ ] _global_active == True?
    _score_3x3
    score > 0 ?
        Yes → _global_active=False,
        score > 0 →

[ ] local_greedy.decide(ctx)
    STAY → _stuck_count=0
    STAY → _stuck_count++
```

```

        _stuck_count >= stuck_threshold(2)?
        Yes → _global_active=True _stuck_count=0
        No → STAY

```

六、迷雾约束验证：AI 是否真的只用 fog_map?

局部贪心

- `_score_3x3` 和 `_cell_score` 只访问 `maze.fog_map[r][c]`
- `is_walkable` 方法: `fog_map[r][c]` is None 时返回 False (未知格视为不可走)
- □ 完全遵守迷雾约束

全局规划

- `_bfs_to_frontier`: BFS 只扩展 `is_walkable == True` 的格子 (跳过 None 格)
- `_plan_path` → `astar` / `dijkstra`: 调用 `maze.is_walkable` 作为可通行判断
- `_scan_boss`: 扫描 `fog_map` 而非 `ground_truth`
- □ 完全遵守迷雾约束

模拟器 (不属于AI, 但确认真确性)

- `ground_truth`: 完整地图, 仅模拟器内部使用, AI 无法访问
 - `_step` 中的墙壁判断用 `ground_truth` (物理碰撞逻辑, 正确)
 - `_reveal_fov`: 移动后将 `ground_truth` 内容写入 `fog_map` (正确揭露)
 - □ 模拟器正确隔离了 AI 视野
-

七、当前已知问题与待修复项

问题1: 空地 score 为负导致 AI 不穿越已知通道 (□ 高优先级)

现象: AI 走到某条通道末端后, 因周围空地 `score = -10`, 而旁边黑雾 `score = +8`, 导致 AI 选择回头去探索黑雾, 而非继续沿通道走向更远的前沿。

根本原因: $(\text{raw_value} - \text{movement_cost}) / \text{dist}$ 中, `step_cost=10` 使任何空地格得分均为 -10。

期望修复方案:

```

#
score = raw_value / dist          #          =0  =50/dist
score -= visited_penalty          #
score -= w_backtrack              #
#  step_cost

```


这样空地 score = 0, 不再是负数, AI 愿意穿越空地继续探索。黑雾 score = explore_bonus = 8, 仍然是正数, AI 优先探索未知区域。

问题2: stuck_threshold=2 过小导致频繁切换全局

现象: 局部贪心在走过金币区域后视野内全是已探索空地, score 全为负, 连续 2 次 STAY 立即切全局, 全局 EXPLORE 阶段规划一条很长的前沿路径, 走完后又切回局部。频繁切换造成路径不连贯。

建议: 问题1修复后, 空地 score=0 (不再触发 STAY), 此问题可能自然消失。

问题3: _score_3x3 中对角黑雾格不纳入探索 (设计限制)

对角黑雾格 (dist=2) 不给探索奖励, 意味着 AI 只会为了直接邻居的黑雾而移动。若某方向的直接邻居是已知空地但其对角是大片黑雾, AI 不会主动去探索。(此问题在问题1修复后有所缓解, 因为空地 score=0 不再阻止移动)

八、配置参数速查 (config.json)

```
{
  "local": {
    "coin_value": 50, //
    "trap_penalty": 30, //
    "step_cost": 10.0, // 1
    "explore_bonus": 8.0, //
    "visited_penalty": 3.0, //
    "w_backtrack": 2.0 //
  },
  "global": {
    "w_coin": 2.0, // _best_coin_target
    "w_trap": 1.5, //
    "retry_buffer": 3, // N
    "min_explore_ratio": 0.4, // RUSH
    "rush_round_threshold": 50, // BOSS
    "trap_step_cost": 31 // Dijkstra >30
  },
  "sim": {
    "max_rounds": 500 //
  },
  "composite": {
    "stuck_threshold": 2 // STAY
  }
}
```